

1. The MERGE-SORT algorithm is a divide-and-conquer sorting algorithm. It repeatedly divides the array into two halves until each subarray contains a single element. The subarrays are then merged in sorted order to produce the final sorted array.

MERGE-SORT(A, p, r)

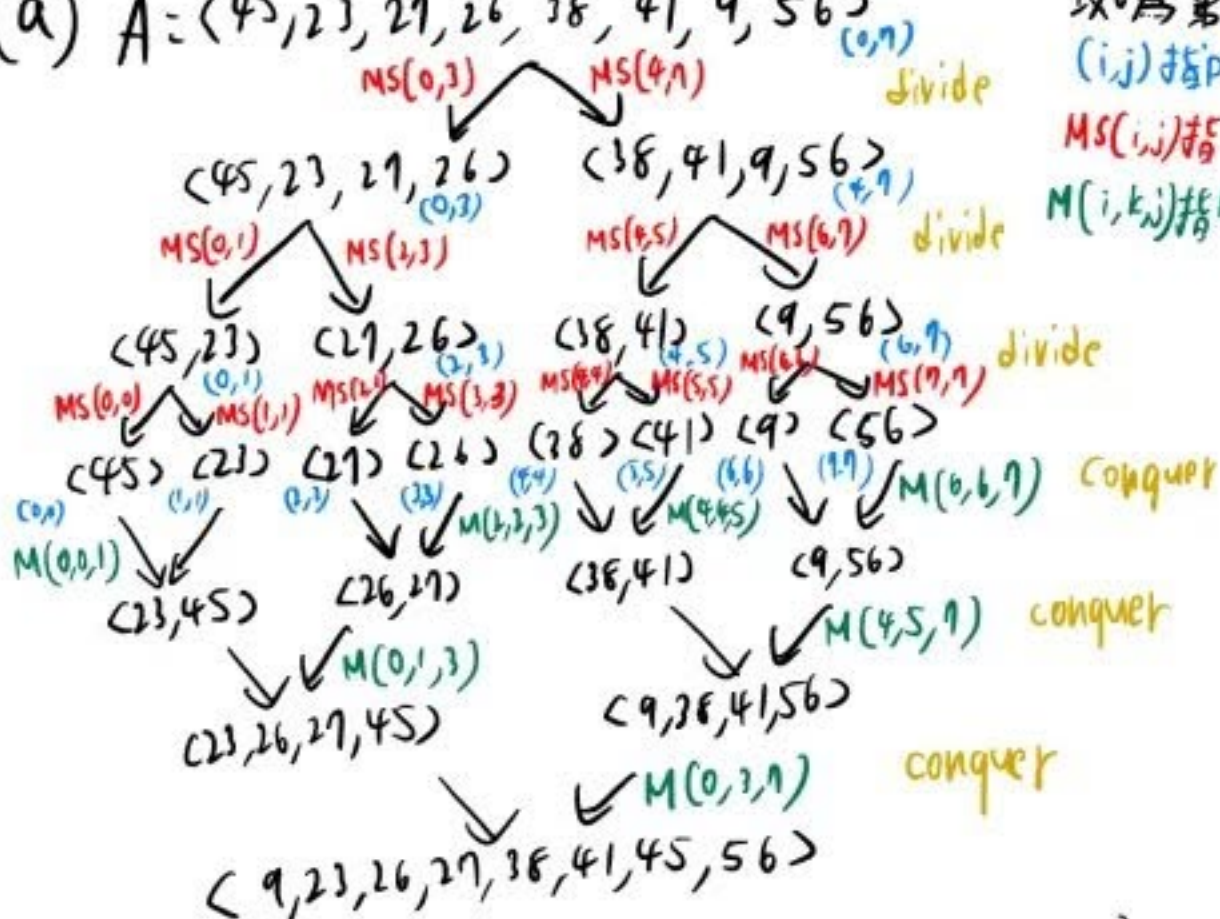
```

1 if  $p < r$ 
2    $q = \lfloor (p+r)/2 \rfloor$ 
3   MERGE-SORT( $A, p, q$ )
4   MERGE-SORT( $A, q+1, r$ )
5   MERGE( $A, p, q, r$ )

```

- (a) Illustrate the operation of MERGE-SORT on the array $A = \langle 45, 23, 21, 26, 38, 41, 9, 56 \rangle$.
 (b) Analyze the time complexity of MERGE-SORT by using the recursion-tree method.
 (c) What is the best-case running time of MERGE-SORT?
 (d) What is the worst-case running time of MERGE-SORT?

(a) $A = \langle 45, 23, 21, 26, 38, 41, 9, 56 \rangle$



以 0 為第 1 個 index

(i, j) 指 $p=i, r=j$

$MS(i, j)$ 指 Merge-Sort(A, i, j)

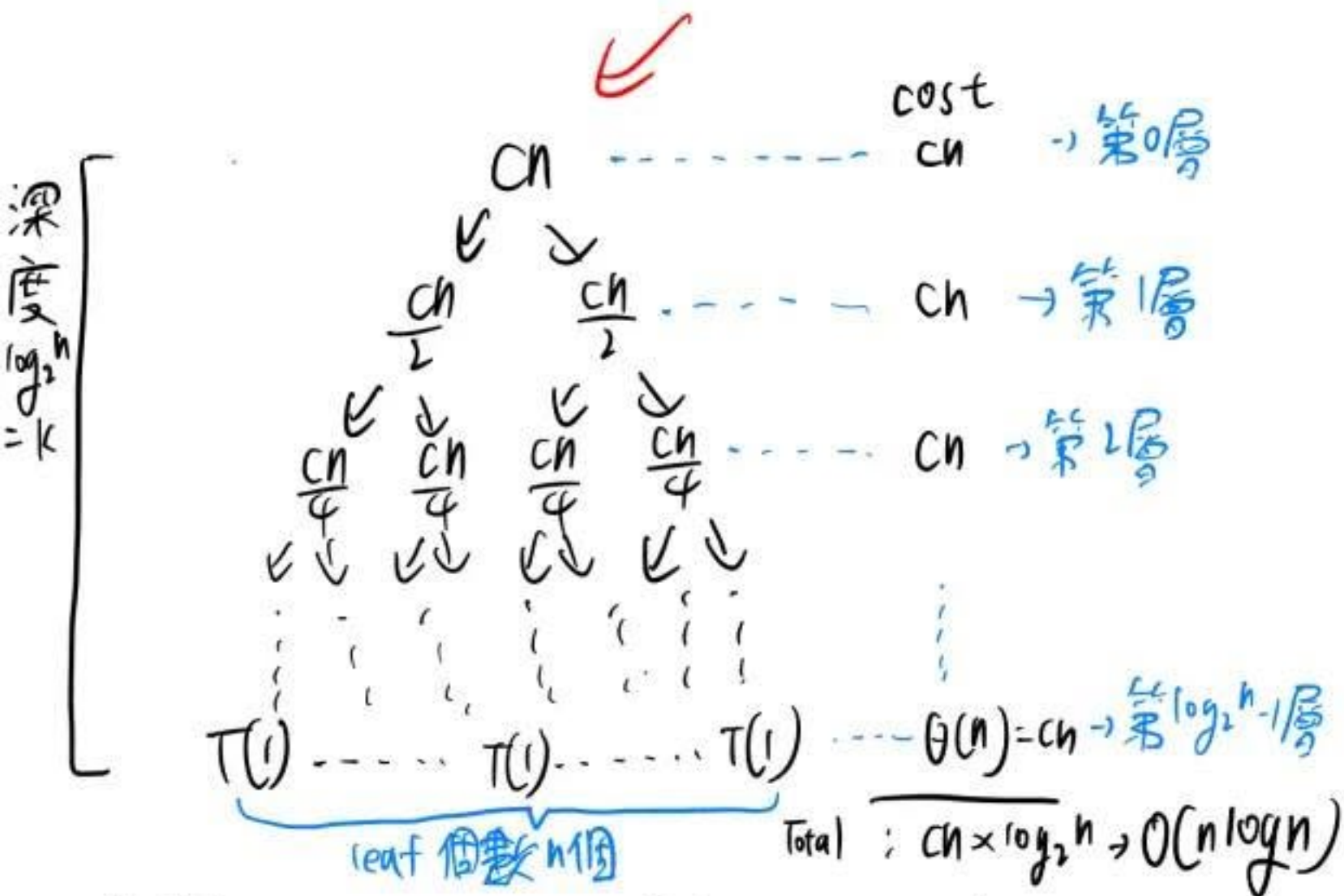
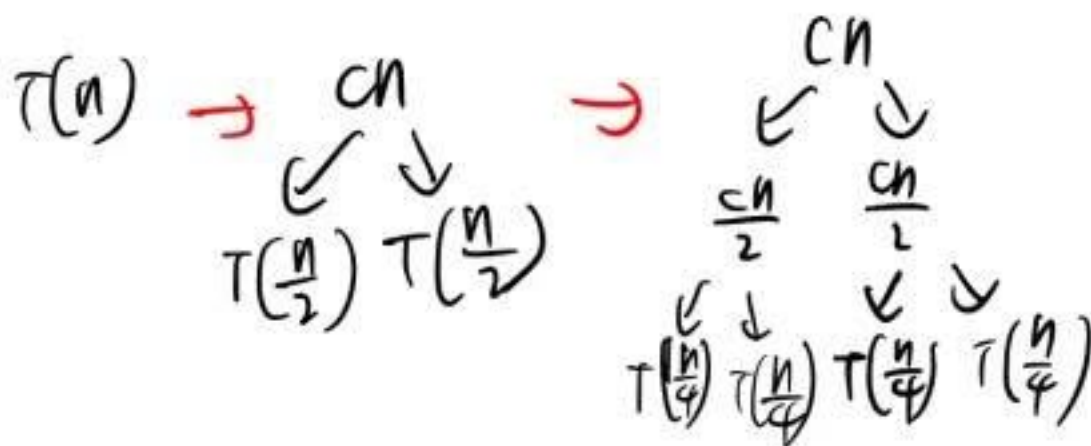
$M(i, k, j)$ 指 Merge(A, i, k, j)

(b) Merge sort 的 running time (設 n 為 array 大小):

$$T(n) = \begin{cases} \theta(1) & \text{if } n=1 \\ T(\lceil \frac{n}{2} \rceil) + T(\lfloor \frac{n}{2} \rfloor) + \theta(n) & \text{if } n>1 \end{cases}$$

忽略邊界條件 (ceil, floor, ...): $T(n) = 2T(\frac{n}{2}) + \theta(n)$

並設 $n = 2^k$, 建造 recursion-tree



	共 h 层				
第 i 层	0	1	2	...	$h-1$
array size	n	$\frac{n}{2}$	$\frac{n}{4}$	...	$\frac{n}{2^{h-1}}$

$$\frac{n}{2^{h-1}} = 1 \Rightarrow n = 2^{h-1} \Rightarrow h = \log_2 n + 1$$

$$\begin{aligned}
 \text{total cost} &= cn(\log_2 n + 1) \\
 &= cn \log_2 n + cn \\
 &= \Theta(n \log_2 n)
 \end{aligned}$$

$$\text{worst-case} = \text{best-case} = \Theta(n \log n)$$